CS221 Problem Workout

Week 3

Key Takeaways from this Week

1. **Machine Learning II**:

   - **Generalization**: The real objective: minimize loss on unseen future examples, not the training loss. Use the test set loss to estimate the performance on unseen examples. Try to minimize training error, but keep the hypothesis class small.
   - **Best Practices**: Follow good data hygiene and always prepare a test set and don't look at it. Use the validation set for tuning and testing. Practice allows you to develop more intuition, learn the best design decisions, and how to tune hyperparameters.
   - **K-means**: simple a widely-used method for discovering cluster structure in data. K means can mean the objective and the algorithm.

2. **Introduction to Search**: we introduced search problems, our first instance of a state-based model. We formalized how to define search problems, with states, successors, actions, and cost functions.

   - **Tree Search**: Four algorithms to keep in mind: backtracking search, depth-first search, breadth-first search, and depth-first search with iterative deepening.
   - **Dynamic Programming**: an algorithm that is akin to backtracking search with memoization with the benefits of exponential savings. The states in DP contain a summary of past actions sufficient to choose future actions optimally.

Practice Problems

1) Problem 1

Sabina has just moved to a new town, which is represented as a grid of locations (see below). She needs to visit various shops $S_1, \ldots, S_k$. From a location on the grid, Sabina can move to the location that is immediately north, south, east, or west, but certain locations have been blocked off and she cannot enter them. It takes one unit of time to move between adjacent locations. Here is an example layout of the town:

	(2,5)	(3,5)	(4,5)	
(1,4)	**S1** (2,4)	(3,4)	**S2** (4,4)	(5,4)
(1,3)	(2,3)		(4,3)	(5,3)
	(2,2)	(3,2)	(4,2)	**S3** (5,2)
House (1,1)	(2,1)	**S4** (3,1)	(4,1)	(5,1)

Sabina lives at $(1, 1)$, and no location contains more than one building (Sabina's house or a shop).

(a) Sabina wants to start at her house, visit the shops $S_1, \ldots, S_k$ **in any order**, and then return to her house as quickly as possible. We will construct a search problem to find the fastest route for Sabina. Each state is modeled as a tuple $s = (x, y, A)$, where (x, y) is Sabina's current position, and A is some auxiliary information that you need to choose. If an action is invalid from a given state, set its cost to infinity. Let V be the set of valid (non-blocked) locations; use this to define your search problem. You may assume that the locations of the k shops are known. You must choose a minimal representation of A and solve this problem for general k. Be precise!

- Describe A: $A = [A_1, A_2, \ldots A_k]$; $A_i \in \{0, 1\}$. whether S_i visited or not.
- $s_{\text{start}} =$ $(1, 1, [0, 0, \ldots 0])$

- Actions$((x, y, A)) = \{\text{N}, \text{S}, \text{E}, \text{W}\}$ $A' = A'_i$ for $i \in [1]$ A'_i: if new location contains shop i else 0

- Succ$((x, y, A), a) =$

$$\begin{cases} (x, y+1, A') & \text{if } a = N \\ (x, y-1, A') & \text{if } a = S \\ (x+1, y, A') & \text{if } a = E \\ (x-1, y, A') & \text{if } a = W \end{cases}$$

- Cost$((x, y, A), a) =$

1 : if $(x', y') \in V$; ∞ . otherwise

$(x', y', A') = \text{Succ}((x, y, A), a)$.

- IsGoal$((x, y, A)) =$

$[x = 1 \wedge y = 1 \wedge A = [1, \ldots, 1]]$.

(b) Sabina is considering a few different methods to visit the shops in as few steps as possible. For each of the following, state whether the algorithm will be able to find a path to visit all shops in as few steps as possible, and if so, provide a running time assuming an $N \times N$ grid.

- Depth-First Search (DFS)
- Backtracking search

DFS: Not! Since Assumption Cost(s, a) = 0

And not be able to find the min-cost path.

DFS'll

Backtracking : Will be able to find min-cost path.

Since : $O(b^D)$; where $b=4$.

states : N^2 (position) 2^k (λ)

$\Rightarrow$ $O(4^{2^k N^2})$.

(c) Recall that Sabina is allowed to visit the shops **in any order**. But she is impatient and doesn't want to wait around for your search algorithm to finish running. In response, you will use the A* algorithm, but you need a heuristic. For each pair of shops (S_i, S_j) where $i \neq j$ and $1 \leq i, j \leq k$, define a **consistent** heuristic $h_{i,j}$ that approximates the time it takes to ensure that shops S_i and S_j are visited and then return home. Computing $h_{i,j}(s)$ should take $O(1)$ time.

Define: $h_{ij}(x, y, A)$ as follows

$$h_{ij}(x,y,A) = \begin{cases} ① & d((x,y),(1,1)) \quad \text{if } A_i=1 \wedge A_j=1 \\ ② & d((x,y),S_j)+d(S_j,(1,1)) \quad \text{if } A_i=1 \wedge A_j=0 \\ ③ & d((x,y),S_i)+d(S_i,(1,1)) \quad \text{if } A_i=0 \wedge A_j= \\ ④ & \min\{ d((x,y),S_i)+d(S_i,S_j)+d(S_j,(1,1)) \\ & \quad d((x,y),S_j)+d(S_j,S_i)+d(S_i,(1,1)) \\ & \quad \text{if } A_i=0 \wedge A_j=0. \end{cases}$$

$\Rightarrow$ Computing $h_{ij}(s)$ is $O(1)$ ... ($d(\ldots) \Rightarrow O(1)$)

Consistent: Claim

$$Cost(s,a) + h_{ij}(succ(s,a)) - h_{ij}(s) \geq 0$$

$$\forall\, s, a, i, j.$$

Assume $cost(x,a)=1$; For ①

$$1 + \underbrace{d(succ(s,a),(1,1)) - d(s,(1,1))}_{1 \text{ or } -1} \geq 0$$

②&③ : $1 + \underbrace{d(succ(s,a),S_j)}_{1 \text{ or } -1} + d(S_j,(1,1)) - \underbrace{d(s,S_j)}_{1 \text{ or } -1} - d(S_i,a,$ ≥ 0

④ : Find out the min ... And ↑ (analogous..)

i or j !

2) Problem 2

In 16th century England, there were a set of $N+1$ cities $C = \{0, 1, 2, \ldots, N\}$. Connecting these cities were a set of bidirectional roads R: $(i, j) \in R$ means that there is a road between city i and city j. Assume there is at most one road between any pair of cities, and that all the cities are connected. If a road exists between i and j, then it takes $T(i, j)$ hours to go from i to j.

Romeo lives in city 0 and wants to travel along the roads to meet Juliet, who lives in city N. They want to meet.

(a) Fast-forward 400 years and now our star-crossed lovers now have iPhones to coordinate their actions. To reduce the commute time, they will both travel at the same time, Romeo from city 0 and Juliet from city N.

To reduce confusion, they will reconnect after each traveling a road. For example, if Romeo travels from city 3 to city 5 in 10 hours at the same time that Juliet travels from city 9 to city 7 in 8 hours, then Juliet will wait 2 hours. Once they reconnect, they will both traverse the next road (neither is allowed to remain in the same city). Furthermore, they must meet in the end in a city, not in the middle of a road. Assume it is always possible for them to meet in a city.

Help them find the best plan for meeting in the least amount of time by formulating the task as a (single-agent) search problem. Fill out the rest of the specification:

- Each state is a pair $s = (r, j)$ where $r \in C$ and $j \in C$ are the cities Romeo and Juliet are currently in, respectively. R_r, R_j r/j roads, respectively
- Actions$((r, j)) =$ $\{(r', j') : (r, r') \in R_r, (j, j') \in R_j\}$
- Cost$((r, j), a) =$ $\max(T(r, r'), T(j, j'))$
- Succ$((r, j), a) =$ (r', j')
- $s_{\text{start}} = (0, N)$
- IsGoal$((r, j)) = \mathbb{I}[r = j]$ (whether the two are in the same city).

(b) Assume that Romeo and Juliet have done their CS221 homework and used Uniform Cost Search to compute $M(i,k)$, the minimum time it takes one person to travel from city i to city k for all pairs of cities $i,k \in C$.

Recall that an A* heuristic $h(s)$ is consistent if

$$h(s) \leq \text{Cost}(s,a) + h(\text{Succ}(s,a)). \tag{1}$$

Give a consistent A* heuristic for the search problem in (a). Your heuristic should take $O(N)$ time to compute, assuming that looking up $M(i,k)$ takes $O(1)$ time. Explain why it is consistent. Hint: think of constructing a heuristic based on solving a relaxed search problem.

$$h((r,j)) = \min_{c \in C} \max\{M(r,c), M(j,c)\} \tag{2}$$

Prove: $h((r,j)) \leq \text{Cost}((r,j),(r',j')) + \min_{c' \in C} \max\{M(r',c'), M(j',c')\}$

$\min_{c \in C} \max\{M(r,c), M(j,c)\} \leq \max(M(r,r'), M(j,j')) + \dots$

Right $\geq M(r,r') + M(r',c)$ | j ✓

$\geq \min_{c \in C} M(r,c)$

$\Rightarrow$ Consistent.